

PREMISE SELECTION IS NOT ENOUGH: ACTION-CONDITIONAL EVIDENCE ALLOCATION FOR LEAN

Anonymous authors

Paper under double-blind review

ABSTRACT

Premise selection is usually treated as a ranking problem: choose useful lemmas, pass them to a Lean prover, and hope reconstruction succeeds. We argue that this view is missing a second intervention. A selected name must still be compiled into the proof interface that can consume it: Hammer facts, HammerCore inputs, simp lemmas, `solve_by_elim` facts, Aesop fact rules, or Aesop simp rules. We introduce ACE, a typed evidence compiler for *action-conditional evidence allocation* in Lean. The core finding is counterintuitive but repeatable: once the candidate names are fixed, small typed portfolios can recover nearly all tested oracle headroom, while learned routers do not yet beat the fixed typed control. In a Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness, the mixed oracle-core/retrieved action grid solves 58/230 kernel-verified goals versus 38/230 for the best single action; a four-retry fixed portfolio reaches 57/230 out of fold. Removing traced proof-core names still leaves a retrieved-only compiler effect: empty Hammer solves 29/230, the best standalone action solves 35/230, and a fixed portfolio reaches the 52/230 typed oracle. To check that the effect is not only a post-hoc fit to the 230-goal matrix, we freeze the retrieved-only four-action portfolio and evaluate two disjoint fresh folds: on 432 replayable holdout goals, empty Hammer solves 30, the best singleton solves 54, the frozen K=4 portfolio solves 67, and the tested-action oracle is 68. Exact Aesop controls further constrain the mechanism: joint oracle-core plus retrieved exposure is best at 38/230, but identity and simp-only controls reach 36/230 and 35/230, so the supported claim is source/interface sensitivity rather than a large pure facts-versus-simps complementarity. ACE is therefore not a new retriever or an adaptive-router victory; it is evidence that premise selection is incomplete until selected names are typed into executable Lean proof actions.

1 INTRODUCTION

Lean hammers and neural theorem provers depend on premise selection: given a proof goal, the system must choose accessible lemmas that make proof search tractable and reconstructable in Lean. Modern systems therefore invest heavily in retrieving better premises before running a backend prover (Yang et al., 2023; Zhu et al., 2025; Gao et al., 2026). This framing is necessary but incomplete. A selected Lean name is not yet an executable proof-search intervention; it must be represented for the tactic that will consume it. The same name may be useful as a Hammer fact, inert as a simp lemma, harmful as an Aesop rule, or useful only after being paired with a different interface.

This interface decision is easy to overlook because it is not the same as ordinary tactic selection. We are not only choosing whether to call Hammer, Aesop, or simplification. We are choosing how a fixed source of evidence should enter each proof mechanism: as facts, simp rules, HammerCore inputs, `solve_by_elim` facts, or multi-channel Aesop programs. Under bounded search, more evidence is also not monotonic. Additional names can improve proof-core coverage while increasing branching, timeouts, or reconstruction ambiguity. Thus a high-recall premise list can still fail if it is compiled into the wrong Lean interface.

We propose ACE, an action-conditional evidence allocator for bounded Lean proof search. Given a replayable theorem context and a candidate set, ACE compiles selected names into typed proof actions and checks each action in Lean. Every comparison starts from the same empty-premise

Hammer attempt and then spends a fixed retry budget. This protocol turns the main question into a reviewer-checkable intervention study: after names have been proposed, how much verified headroom comes from assigning them to the right proof interface?

The empirical answer is surprisingly strong for simple typed portfolios. In a Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness, we identify 230 replayable goals from 490 cleaned traced-corpus goals. A mixed source pool of traced oracle-core names and learned retrieved candidates gives a typed action oracle of 58/230, compared with 38/230 for the best single action. A four-retry fixed typed portfolio after `hammer_empty` reaches 57/230 out of fold and 58/230 when fit on all training data. Homogeneous $K=4$ controls are much weaker: Aesop-only reaches 49/230, HammerCore-only and simplification-only reach 41/230, and Hammer-only reaches 32/230. The effect is therefore typed diversity, not simply more calls to one backend.

We then remove the most obvious source of concern: traced proof-core assistance. In a retrieved-only anchor on the same 230 replayable goals, empty Hammer solves 29/230 and the best standalone action solves 35/230, while a fixed typed portfolio reaches the 52/230 typed oracle out of fold. To test whether this is a post-hoc peculiarity of the 230-goal matrix, we freeze the retrieved-only four-action portfolio and evaluate two fresh disjoint folds with zero overlap against the original goals. Across 432 replayable holdout goals, empty Hammer solves 30, the best singleton solves 54, $K=3$ and $K=4$ solve 67, and the tested-action oracle is 68. This does not make ACE a full theorem-proving leaderboard system, but it does give prospective evidence that typed compilation survives beyond the matrix used to discover it.

The mechanism is also narrower than our early experiments suggested. Aesop with oracle-core plus retrieved evidence exposed jointly as facts and simp lemmas proves 38/230 goals, compared with 29/230 for empty Aesop. Exact controls show that this is not a large isolated facts-versus-simps complementarity: identity exposure proves 36/230, sims-only proves 35/230, facts-only proves 32/230, retrieved-only joint exposure proves 34/230, and oracle-core-only joint exposure proves 35/230. The robust readout is that selected evidence has source-dependent and interface-dependent behavior; the paper does not need the stronger channel-complementarity claim.

This paper makes three contributions. First, we formulate action-conditional evidence allocation as a distinct layer after premise retrieval: selected names must be typed into Lean proof interfaces, not only ranked. Second, we implement ACE in a real Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness and show that fixed typed portfolios recover nearly all tested oracle headroom in a mixed-source matrix, a retrieved-only anchor, and frozen fresh-holdout folds. Third, we report the boundaries that keep the claim honest: broad insertion can hurt, strict syntactic filtering does not explain the gains, exact Aesop controls weaken a tempting complementarity story, and logistic/ComplementNB allocators do not yet beat the fixed typed control.

The scope is deliberately narrower than a full theorem-proving leaderboard. The retrieved-only and fresh-holdout experiments test the downstream compiler once candidate names are fixed; they are not official reproductions of LeanSearch v2 or another complete global retrieval system. We also distinguish bridge replay from open-ended proof generation. These boundaries sharpen rather than weaken the claim: premise selection is incomplete until selected evidence is compiled into the Lean interface that can actually use it.

2 RELATED WORK

Premise selection and hammers. Premise selection is a long-standing interface between theorem provers and large formal libraries. LeanDojo introduced retrieval-augmented theorem proving infrastructure in Lean (Yang et al., 2023); recent Lean-hammer work studies learned premise ranking and ATP integration (Zhu et al., 2025); and LeanSearch v2 focuses on global Lean 4 premise retrieval (Gao et al., 2026). These systems motivate our setting: a retriever proposes useful names before proof search. ACE studies the next decision after retrieval: how those names should be represented for Lean’s consuming interface.

Proof-action control and feedback. Tactic-selection systems such as TacticToe choose tactics or tactic sequences (Gauthier et al., 2021), while recent Lean systems use compiler or verifier feedback for proof repair, agentic search, or process supervision (Wang et al., 2026; Ospanov et al., 2025; Ma

et al., 2026; Kim & Yun, 2026; Huang et al., 2025; Hu et al., 2024). ACE is narrower: it does not edit proof text or propose new theorem-level plans. It fixes a candidate evidence source and tests the typed action by which selected names enter Hammer, HammerCore, simplification, `solve_by_elim`, and Aesop.

3 PROBLEM FORMULATION

Action-conditional evidence allocation under a fixed retry budget. Let g be a Lean goal and let $\mathcal{A}(g)$ be the accessible name set induced by imports, namespaces, declarations, and local context. A typed proof action is a pair $a = (\tau, P)$, where τ is a Lean interface such as Hammer, HammerCore, simplification, `solve_by_elim`, or Aesop, and $P \subset \mathcal{A}(g)$ is the set of names exposed through that interface. More generally, a typed evidence program is an assignment

$$z_{p,\tau} \in \{0, 1\}, \quad \sum_{p \in \mathcal{A}(g)} z_{p,\tau} \leq b_\tau,$$

where $z_{p,\tau} = 1$ means that name p is exposed to interface channel τ under channel budget b_τ . The same name may receive multiple interface labels: an Aesop joint facts+simps action is one compiled program, not two independent retries. A compiler $C(z)$ maps this assignment to Lean tactic syntax, such as Hammer facts, HammerCore inputs, simplifier lemmas, `solve_by_elim` facts, Aesop safe facts, and Aesop simp rules. This separates four decisions that are often conflated: candidate provenance, channel eligibility, channel assignment, and outer action scheduling.

An evidence allocator chooses a sequence of typed programs,

$$a_t = (\tau_t, P_t), \quad |P_t| \leq k_t,$$

and runs each action under a search budget B_t . The prover returns either a kernel-verified success or a failure observation F_t , such as a timeout, missing premise signal, type mismatch, typeclass failure, failed simplification, or reconstruction failure. A closed-loop selector is a policy

$$a_{t+1} = \pi(g, a_{\leq t}, F_t, H_t),$$

where H_t includes previous attempt history and candidate features. The objective is to maximize verified success under a fixed total retry budget, rather than to maximize static premise recall at a single k . A fixed typed portfolio is one allocator; a learned action router is another. The experiments ask whether interface typing itself creates headroom before claiming that learned routing has solved the allocation problem.

Evaluation axes. We separate three notions that are often conflated. *Kernel-verified proof-action success* means that a patched Lean file containing a candidate proof action checks successfully. This is the main verified metric for ACE. *Trace-core success* means that the selected premise set covers the traced proof-core premises for a goal; it is a premise-recovery metric, not a full proving metric. *Bridge verified success* is stricter than trace-core success: trace-core recovery must coincide with successful replay of the original traced tactic script in a real Lean process. Bridge verification is still not a full proof-generation benchmark, because it replays known tactic scripts, but it filters out trace-core cases that drift too far from Lean execution.

4 METHOD

4.1 OVERVIEW

ACE treats a selected name as incomplete until it is assigned to a proof-action interface. Figure 1 gives the full loop. The system first builds a candidate pool and runs a shared empty-premise Hammer attempt. It then spends a fixed retry budget on typed actions that expose selected names as Hammer facts, HammerCore inputs, simp lemmas, `solve_by_elim` facts, or Aesop facts/simp rules. The important design choice is that the retry budget is spent on compiling evidence into interfaces, not on blindly expanding a single untyped premise list.

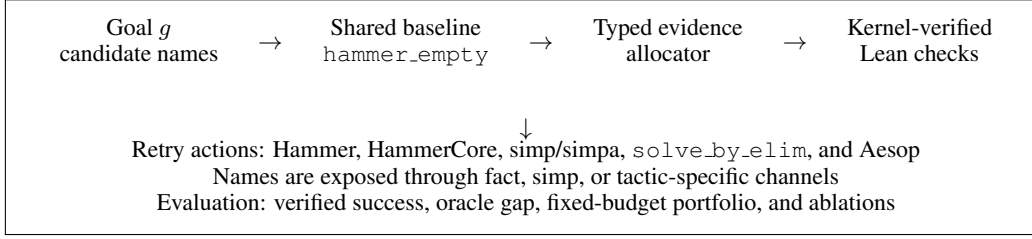


Figure 1: ACE evaluates action-conditional evidence allocation under a fixed retry budget. Selected names are compiled into typed Lean interfaces, and each routed action is checked by the Lean kernel.

4.2 TYPED EVIDENCE PROGRAMS

The allocator is built from action families that consume selected names differently. Hammer actions pass names as ordinary premises; HammerCore actions expose a smaller internal proof-search interface; simplification actions use selected names as rewrite/simp lemmas; solve_by_elim uses names as local facts; Aesop actions can add names as fact rules, simp rules, or both. For each replayable theorem context, we materialize a patched pre-theorem Lean file for each action and classify the result by running Lean with a fixed timeout.

The fixed portfolio is chosen greedily on the training split and evaluated out of fold. Every policy receives the same first action, hammer_empty, and then a retry budget K . This protocol makes the strongest control explicit: an adaptive policy is only useful if it beats or compresses the best fixed sequence of typed actions under the same Lean-call budget. In the current matrix, the train-fitted four-action portfolio after hammer_empty is aesop_core_plus_learned, hammerCore_core_plus_learned, hammer_core_plus_learned16, and solve_by_elim_core.

This design also prevents a common misreading of the method. The fixed portfolio is not intended to prove that a hand-written order is optimal; it is the hard control that any learned action router must beat. Its near-oracle performance is therefore evidence that interface typing itself is a large effect, while the absence of an adaptive win is reported as a boundary rather than hidden as a negative result.

4.3 AESOP FACT/SIMP EXPOSURE

Aesop is the most important new interface in the verified matrix, so we isolate how selected names enter it. Given a ranked name list, we split available names into fact candidates and simp candidates, then instantiate Aesop in three exposure modes: facts-only, simps-only, and facts+simps. We also vary source pools: traced oracle-core names, retrieved-only names, and oracle-core plus retrieved names at budgets 8, 16, and 32. This gives a matched source/budget control: the goal and candidate source are fixed, while the channel assignment is removed, added, or widened.

The Aesop ablation is not a new oracle-improving action set; its role is explanatory. The original broad ablation suggested a strong facts+simps interaction, but exact matched controls give a more conservative readout. For the best current oracle-core plus retrieved source, joint exposure recovers the 38/230 Aesop result, yet identity exposure reaches 36/230, simps-only reaches 35/230, facts-only reaches 32/230, and joint-only over exact single-channel controls is only 2 goals. Retrieved-only joint exposure still reaches 34/230 and oracle-core-only reaches 35/230. The supported mechanism is therefore source/exposure/search sensitivity, not a clean causal claim that facts and simps are complementary in isolation.

4.4 WHY TYPED ACTIONS SHOULD HELP

Let $Y_g(z; B) \in \{0, 1\}$ denote the kernel-verified potential outcome for goal g when typed assignment z is checked under search budget B . The marginal intervention effect of exposing name p through interface τ is

$$\Delta_g(p, \tau \mid z, B) = Y_g(z + e_{p, \tau}; B) - Y_g(z; B).$$

Table 1: Core verified evidence. “Portfolio” is the fixed typed retry sequence after the shared `hammer_empty` call; the fresh-holdout row uses the frozen retrieved-only sequence and an oracle over the predeclared tested actions.

Setting	Empty	Best single	Portfolio	Oracle	Readout
Mixed-source 230	29/230	38/230	57/230 OOF	58/230	typed actions create +20 oracle headroom
Retrieved-only 230	29/230	35/230	52/230 OOF	52/230	removes traced proof-core assistance
Frozen fresh holdout	30/432	54/432	67/432	68/432	transfers beyond the discovery matrix

Setting	Single interface	Small typed portfolio	Tested oracle
Mixed-source 230	38	57	58
Retrieved-only 230	35	52	52
Fresh holdout 432	54	67	68

Figure 2: Result ladder. Across the mechanism matrix, deployability anchor, and frozen holdout, most tested oracle headroom is recovered by a small typed portfolio rather than by the strongest single interface.

This formulation is intentionally intervention-level rather than only a marginal retrieval score. It permits the same name to have positive effect in one interface and negative effect in another, allows facts and simp rules to interact through the current program z , and permits larger evidence sets to have negative marginal effect under bounded search. A typed action helps when the compiler finds assignments whose realized intervention effects are positive under the Lean backend’s budget, not merely when a retriever ranks relevant names highly.

5 EXPERIMENTS

Protocol. All main results are kernel-verified Lean checks in a Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness. We first clean 490 traced-corpus goals and keep the 230 goals whose original tactic script replays before the theorem statement. We then run typed proof actions in Lean and count a success only when the patched file checks. To separate interface allocation from traced proof-core assistance, we repeat the budgeted compiler test with only retrieved candidates. Finally, we freeze the retrieved-only action subset and evaluate two disjoint 500-goal folds with zero overlap against the original 230-goal set; the folds contain 432 replayable goals in total. Trace-core and bridge experiments are retained in Appendix 10 as discovery evidence for bounded retries, not as the main proving metric.

Table 1 is the main empirical result. On the mixed-source 230-goal matrix, the best single typed action solves 38 goals, while the typed action oracle solves 58; a four-retry fixed portfolio reaches 57 out of fold. Removing traced proof-core names gives the retrieved-only anchor: the best standalone action solves 35/230, but the fixed typed portfolio reaches the 52/230 typed oracle. The frozen holdout check gives the same readout prospectively: on 432 replayable goals, the frozen $K=4$ sequence solves 67 goals, compared with 54 for the best singleton and 68 for the tested-action oracle. Thus the main effect is not a post-hoc fit to one matrix and not only an oracle-core artifact.

Table 2 shows why the fixed sequence is a strong control rather than a weak baseline. In all three settings, the portfolio approaches the tested oracle quickly. On the retrieved-only anchor, $K=3$ already reaches the oracle. On the frozen holdout, $K=3$ and $K=4$ recover 67/68 tested-action opportunities, and the strict after-`hammer_empty` coverage is 37/38. The remaining one-goal gap is useful because it shows that the frozen policy is not evaluated against a trivial oracle.

Table 3 is the prospective check that matters most for presentation. The fold-level numbers are not identical, but both move in the same direction: the frozen typed sequence beats the strongest singleton, and the combined result is within one goal of the tested-action oracle. This is the main reason the paper can present fixed typed diversity as a real compiler effect rather than a training-set artifact.

Table 2: Budget path after the shared `hammer_empty` call. The same typed-portfolio story holds in the mixed-source matrix, the retrieved-only anchor, and the frozen holdout.

Setting	K=1	K=2	K=3	K=4	Oracle
Mixed-source 230	49/230	55/230	55/230	57/230	58/230
Retrieved-only 230	47/230	50/230	52/230	52/230	52/230
Frozen holdout 432	56/432	58/432	67/432	67/432	68/432

Table 3: Fresh-holdout fold details. The action subset and order are frozen before evaluation; the two folds have zero overlap with the original 230-goal matrix.

Fold	Replayable	Empty	Best single	K=3	K=4	Oracle
Fold 0	208	16	23	28	28	28
Fold 1	224	14	31	39	39	40
Combined	432	30	54	67	67	68

Table 4 rules out a simple “more retries” explanation. Four Aesop retries are useful but remain eight goals behind the typed portfolio; HammerCore-only and simplification-only controls are much weaker; and Hammer-only barely improves over empty Hammer. The portfolio is strong because it crosses proof interfaces.

Table 5 turns a negative result into a boundary on the claim. The learned gates see goal text, first-failure status, and typed fact/simp pool features, but they do not surpass the fixed typed control. This prevents an overclaim about adaptive routing and strengthens the paper’s simpler message: the immediate missing layer after retrieval is typed compilation.

Table 6 keeps the mechanism claim calibrated. Aesop is an important interface, and joint exposure is the best Aesop row, but the gap over identity and simps-only controls is modest. This is why the paper’s main claim is typed evidence allocation across interfaces, not a specialized claim that Aesop facts and simps form a large isolated complementarity.

Table 7 explains why we present ACE as a typed evidence compiler rather than as a generic portfolio or adaptive-router method. Repeating one family is weaker than mixing interfaces; learned routers do not yet compress the fixed control; Aesop exposure is useful but not a standalone explanation; and broad filtering does not recover the effect. These negative and diagnostic results are useful because they make the supported claim sharper: after a retriever proposes names, Lean still needs a typed decision about how those names enter proof search.

6 THEORY AND ANALYSIS

Proposition 1 (A scalar premise score cannot represent typed intervention value). *Let $Y_g(z; B) \in \{0, 1\}$ be the kernel-verified outcome for goal g under typed evidence assignment z and search budget B . If there exist a name p and two interfaces τ_1, τ_2 such that*

$$Y_g(z + e_{p, \tau_1}; B) \neq Y_g(z + e_{p, \tau_2}; B),$$

then no interface-agnostic premise score for p can represent the full intervention effect of exposing p to Lean.

Sketch. The intervention $e_{p, \tau}$ changes both the syntax sent to Lean and the backend search process that consumes the name. If two interface assignments yield different verified outcomes under the same goal and budget, a premise ranking over p alone has discarded information needed to choose the successful action. This is the formal reason for compiling selected evidence into typed proof actions rather than treating retrieval as the final decision. Table 1 gives the empirical version: the best single action is far below the typed oracle, while a small typed portfolio recovers nearly all tested headroom.

Proposition 2 (Premise coverage is not monotonic in premise budget). *There can exist premise sets $P_1 \subset P_2$ such that P_1 covers enough useful premises for the backend to succeed under budget B , while P_2 contains the same useful premises but causes timeout or reconstruction failure under the same budget.*

Table 4: Homogeneous K=4 controls under the same outer Lean-call budget on the mixed-source 230-goal matrix. Typed diversity beats repeated use of one proof-action family.

Portfolio family	OOF K=4	Strict goals
Full typed grid	57/230	28/29
Aesop-only	49/230	20/29
HammerCore-only	41/230	12/29
Simplification-only	41/230	12/29
<code>solve_by_elim</code> -only	34/230	5/29
Hammer-only	32/230	3/29
Raw arithmetic/normalization	29/230	0/29

Table 5: Learned allocator gate on the mixed-source 230-goal matrix. Learned routers match the fixed control at high budgets but do not beat or compress it.

K	Fixed OOF	Pure logreg	Residual logreg	Residual CNB
1	49/230	41/230	41/230	38/230
2	55/230	41/230	55/230	52/230
3	55/230	44/230	56/230	55/230
4	57/230	47/230	57/230	57/230

Sketch. Adding premises preserves proof-core coverage but can increase branching, generated clauses, simplification search, or reconstruction ambiguity. Under fixed time/search budget, the prover may fail on the larger set even though the smaller set contains the same proof-relevant lemmas. The boundary checks in Table 7 are consistent with this view: broad Aesop insertion and strict syntactic filtering do not explain the gains, while typed diversity does.

7 SCOPE AND DEPLOYMENT NOTES

The experiments are designed as a proof-action intervention study rather than a leaderboard entry. The main matrix uses 230 replayable pre-theorem Mathlib contexts, and the frozen holdout adds 432 replayable retrieved-only contexts with no overlap against the discovery set. This setup avoids target-theorem leakage and makes each reported success a kernel-checked Lean result. A full external deployment would naturally combine ACE with stronger global retrievers such as LeanSearch-style systems; our retrieved-only rows study the downstream compiler once such candidates have been proposed.

The current strongest policy is intentionally simple: a fixed typed portfolio. This is a useful property for the present claim because it shows that much of the headroom comes from interface typing itself, before more complex adaptive routing is needed. Learned routers, trace-core controllers, wallclock frontiers, and bridge diagnostics are therefore reported as supporting analyses rather than as the paper’s central promise.

8 DISCUSSION

What changes for Lean premise systems? The practical implication is that a premise selector should not hand a prover only an ordered list of names. It should hand off a typed evidence program: which names are ordinary Hammer facts, which are HammerCore inputs, which are simplification lemmas, which are elimination facts, and which are Aesop fact or simp rules. This is a small interface change but a different abstraction boundary. Retrieval remains responsible for proposing useful names; the compiler layer decides how those names become executable proof actions.

Why the fixed portfolio is informative. The fixed portfolio is not presented as an optimal hand-written strategy. It is the strongest failure-agnostic control we found under the same outer retry budget. Its near-oracle behavior is exactly what makes the result interesting: much of the current headroom is recovered by crossing typed interfaces at all, before asking a learned router to decide per goal. This

Table 6: Exact Aesop source/channel controls. Joint exposure is best, but the matched controls support source/interface sensitivity rather than a large pure facts-versus-simps complementarity.

Aesop control	Verified goals	Readout
empty Aesop	29/230	default baseline
retrieved-only, joint	34/230	non-oracle source helps
oracle-core-only, joint	35/230	oracle source alone is similar
oracle-core+retrieved, joint	38/230	best Aesop row
oracle-core+retrieved, identity	36/230	identity explains much of the lift
oracle-core+retrieved, simps-only	35/230	simp exposure is strong
oracle-core+retrieved, facts-only	32/230	fact exposure is weaker

Table 7: Focused boundary checks. These controls rule out the main shortcuts while keeping the paper’s claim narrow: the supported mechanism is typed interface allocation, not a learned-router or pure Aesop-channel claim.

Question	Control result	Readout
Is this just more calls to one backend?	Full typed K=4: 57/230; Aesop-only: 49/230; HammerCore/simp-only: 41/230; Hammer-only: 32/230.	Interface diversity matters.
Is learned routing the contribution?	Best logistic/ComplementNB gates match but do not beat fixed K=4: 57/230.	Fixed portfolio is the hard control.
Is Aesop facts+simps the whole story?	Joint oracle-core+retrieved: 38/230; identity: 36/230; simps-only: 35/230; facts-only: 32/230.	Source/interface sensitivity, not a large pure complementarity.
Does syntactic cleanup explain the gains?	Strict-filtered oracle: 4/230; combined oracle remains 58/230.	Cleanup alone is not the mechanism.
What is the call/time cost?	hammer_empty: 29/230 at 1.00 calls, 8.31s; train-fitted K=4: 58/230 at 4.18 calls, 40.07s.	Main comparisons are matched by Lean calls; wallclock is transparent.

also sets a higher bar for future adaptive methods. A router should not be compared only to empty Hammer or to a single tactic; it should beat a compact typed portfolio.

A compiler checklist for future systems. The experiments suggest a simple reporting and implementation checklist for retrieval-augmented Lean provers. First, report the candidate source separately from the proof-action interface; otherwise a retrieval failure and an interface-compilation failure are indistinguishable. Second, include a fixed typed portfolio as a static control before claiming that an adaptive router is useful. Third, report a retrieved-only version of the compiler test when traced proof-core names are available during development. Fourth, report whether extra names are helping the backend or merely increasing search and reconstruction cost.

Why this checklist is conservative. The checklist does not require a new benchmark or a new global retriever. It can be applied to any retrieval source that proposes Lean names: a learned retriever, a lexical retriever, a search-based retriever, or a human-provided candidate list. That makes ACE a modular compiler layer. Its job is not to decide which names are globally relevant; its job is to make a selected candidate set executable under the proof interfaces that Lean actually provides.

Why this is not only a benchmark result. The main result is an intervention on the proof interface, not a new leaderboard score. The same candidate source can produce different verified outcomes depending on how names enter Lean. This creates a reusable diagnostic for future prover work: when a retriever appears weak, one should ask whether the missing step is retrieval quality or evidence compilation. Conversely, when a large premise set hurts, the remedy may be a different typed action rather than another reranking pass over the same untyped list.

How to read the remaining failures. The unsolved goals are also informative because they are partitioned by a verified intervention protocol. A miss after empty Hammer and after every single typed action is different from a miss after the full portfolio; the former points to candidate quality,

Table 8: How each evidence block supports the main claim. The rows are ordered by the role they play in the argument, not by when the experiments were run.

Role in the argument	Evidence used in the main text	What it rules out
Interface effect exists	58/230 typed oracle vs 38/230 best single; 57/230 fixed OOF.	The result is not just one strong tactic.
Not trace-core leakage	Retrieved-only portfolio reaches 52/230 vs 35/230 best single.	The effect does not require traced proof-core names.
Not post-hoc overfit	Frozen holdout reaches 67/432 vs 54/432 best singleton.	The portfolio is not only fit to the discovery matrix.
Not repeated backend calls	Homogeneous K=4 controls are below full typed K=4.	The gain comes from typed diversity.
Not an adaptive-router claim	Learned gates match but do not beat fixed K=4.	The paper’s contribution is the typed compiler layer.

Table 9: Practical checklist induced by the experiments. Each row corresponds to a common ambiguity in retrieval-augmented proving.

Question for a Lean prover	Recommended measurement	Why it matters
Did retrieval or interface compilation fail?	Separate candidate source, typed action, and verified outcome.	A good name list can still fail under the wrong interface.
Is a learned router truly useful?	Compare against the best fixed typed portfolio at the same K.	Empty Hammer and single actions are too weak as controls.
Does the effect depend on proof traces?	Run a retrieved-only compiler anchor.	This separates deployable candidate use from oracle-core assistance.
Does more evidence help or hurt?	Report strict goals, homogeneous families, and wallclock.	Premise coverage and proof-search tractability can diverge.

while the latter points to missing proof actions, longer reasoning chains, or backend search limits. This is why the paper reports typed oracles, homogeneous controls, strict-goal filters, and retrieved-only anchors together. They turn a raw failure count into a map of where the prover stack should improve next. In that sense, ACE is useful even when it does not solve a goal: it makes the failure mode more local than an untyped premise-ranking metric can.

What a stronger next system should do. The natural next step is to learn an allocator that beats the fixed typed portfolio while preserving the same accounting discipline: same first attempt, same retry budget, same verified-outcome metric, and the same strong static controls. The present experiments make that target concrete. They identify where the oracle headroom is, show that a simple typed sequence captures most of it, and separate the compiler effect from both traced proof-core assistance and post-hoc portfolio fitting.

9 CONCLUSION

This paper argues that Lean premise selection is incomplete without action-conditional evidence allocation. ACE implements this view as a typed evidence compiler over Hammer, HammerCore, simplification, `solve_by_elim`, and Aesop interfaces. In the mixed-source 230-goal matrix, the typed action grid reaches 58/230 verified proofs versus 38/230 for the best single action, and a four-retry fixed typed control recovers 57/230 out of fold. In the retrieved-only anchor, the same compiler view reaches 52/230 with a fixed portfolio, compared with 29/230 for empty Hammer and 35/230 for the best standalone action. In two frozen fresh-holdout folds, the predeclared K=4 portfolio solves 67/432 replayable goals versus 54/432 for the best singleton and 68/432 for the tested-action oracle. The exact Aesop controls make the mechanism more precise: source and interface assignment matter, but a large pure facts-versus-simps complementarity is not supported. The result is a narrow but concrete lesson for Lean provers: retrieval should hand off not just a list of names, but a typed evidence program that tells Lean how those names should enter proof search.

REFERENCES

- Guoxiong Gao, Zeming Sun, Jiedong Jiang, Yutong Wang, Jingda Xu, Peihao Wu, Bryan Dai, and Bin Dong. Leansearch v2: Global premise retrieval for lean 4 theorem proving. *arXiv preprint arXiv:2605.13137*, 2026. URL <https://arxiv.org/abs/2605.13137>.
- Thibault Gauthier, Cezary Kaliszyk, Josef Urban, Ramana Kumar, and Michael Norrish. Tactictoe: Learning to prove with tactics. *Journal of Automated Reasoning*, 65:257–286, 2021. URL <https://arxiv.org/abs/1804.00596>.
- Edward J. Hu et al. minictx: Neural theorem proving with long contexts. *arXiv preprint arXiv:2408.03350*, 2024. URL <https://arxiv.org/abs/2408.03350>.
- Suozhi Huang, Peiyang Song, Robert Joseph George, and Anima Anandkumar. Leanprogress: Guiding search for neural theorem proving via proof progress prediction. *arXiv preprint arXiv:2502.17925*, 2025. URL <https://arxiv.org/abs/2502.17925>.
- Minsu Kim and Se-Young Yun. Process-verified reinforcement learning for theorem proving via lean. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=P00k4DFaXF>.
- David Ma, Kaijing Ma, Shawn Guo, Yunfeng Shi, Enduo Zhao, Jiajun Shi, Zhaoxiang Zhang, Gavin Cheung, Jiaheng Liu, and Zili Wang. Oprover: A unified framework for agentic formal theorem proving. *arXiv preprint arXiv:2605.17283*, 2026. URL <https://arxiv.org/abs/2605.17283>.
- Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. Apollo: Automated llm and lean collaboration for advanced formal reasoning. In *Advances in Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=fxDCgOruk0>.
- Evan Wang, Simon Chess, Daniel Lee, Siyuan Ge, Ajit Mallavarapu, and Vasily Ilin. Learning to repair lean proofs from compiler feedback. *arXiv preprint arXiv:2602.02990*, 2026. URL <https://arxiv.org/abs/2602.02990>.
- Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.15626>.
- Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. Premise selection for a lean hammer. *arXiv preprint arXiv:2506.07477*, 2025. URL <https://arxiv.org/abs/2506.07477>.

A ADDITIONAL EXPERIMENTAL DETAILS

A.1 CANONICAL OUTPUT SOURCES

All numbers in the main tables are copied from canonical JSON or Markdown outputs in the project directory:

- Verified typed action grid: `outputs/mathlib430_pretheorem_action_matrix_scaled230_aesop_ablation_merged.md`.
- Budgeted typed portfolio: `outputs/mathlib430_budgeted_action_policy_scaled230_aesop_ablation.md`.
- Typed allocator gate: `outputs/mathlib430_typed_allocator_gate_scaled230_aesop_ablation.md`.
- Retrieved-only anchor summary: `analysis/mathlib430_retrieved_only_anchor_summary.md`.
- Retrieved-only anchor matrix: `outputs/mathlib430_pretheorem_action_matrix_scaled230_retrieved_only_anchor_merged.md`.

- Retrieved-only fixed stability: `outputs/mathlib430_fixed_portfolio_stability_scaled230_retrieved_only_anchor.md`.
- Frozen fresh-holdout summary: `analysis/mathlib430_fresh_holdout_summary.md`.
- Fresh-holdout fold summaries: `analysis/mathlib430_fresh_holdout_fold0_summary.md` and `analysis/mathlib430_fresh_holdout_fold1_summary.md`.
- Fresh-holdout merged action outputs: `outputs/mathlib430_fresh_holdout_fold0_frozen_actions_merged.md` and `outputs/mathlib430_fresh_holdout_fold1_frozen_actions_merged.md`.
- Exact Aesop source/channel controls: `analysis/mathlib430_aesop_exact_controls_summary.md`.
- Initial broad Aesop fact/simp ablation: `analysis/mathlib430_aesop_ablation_scaled230.md`.
- Initial Aesop counterfactual channel control: `analysis/mathlib430_aesop_counterfactual_controls.md`.
- Evidence-contract audit and homogeneous controls: `analysis/mathlib430_evidence_contract_audit.md`.
- Strict interface filtering boundary: `analysis/mathlib430_el_strict_interface_filtering.md`.
- Strict interface taxonomy: `analysis/mathlib430_strict_action_taxonomy_scaled230_aesop_ablation.md`.
- Local trace-core: `outputs/phase1_mathlib_trace_core_2000_feature_ablation_a40.json`.
- Phase 1 bridge: `outputs/phase1_reconstruction_bridge_report_100.md`.
- Timeout stress: `outputs/phase2_timeout_stress_2000_a40.json`.
- Imported-core heldout: `outputs/phase3_second_stage_controller_eval_500_a40.json` plus final guardrail deltas.
- Split stability: `outputs/phase3_split_stability_summary_4x500.md`.
- Imported bridge report: `outputs/phase3_bridge_report_200.md`.
- Imported bridge taxonomy: `outputs/phase3_bridge_failure_taxonomy_200.md`.
- Imported bridge inspection: `outputs/phase3_bridge_negative_case_inspection_200.md`.

A.2 TRACE-CORE DISCOVERY SUMMARY

Table 10: Representative trace-core discovery results that motivated the verified typed-action protocol. These rows are not the main proving metric; they explain the design pressure for bounded retries, non-monotonic premise budgets, and bridge checks.

Discovery question	Strongest useful result	Readout for current paper
Can failure feedback improve local premise recovery?	Local trace-core FAR-HAMMER reaches 95.7% trace success and 72.0% bridge verified success; blind top- k expansion reaches 91.5% trace and 43.0% bridge.	Failure events can change which evidence is useful, but bridge verification is stricter than trace-core recovery.
Can adding more premises hurt?	Timeout-conditioned shrink reaches 69.3% success with 23.5% timeout rate, while equal-budget top- k falls to 43.8% with 56.2% timeouts.	Premise coverage is not monotonic under bounded search, motivating typed budgets instead of broad insertion.
Do local heuristics solve global imported premises?	Imported-core lexical/BM25 stress stays near 12% success before learned retrieval.	Local feature control is insufficient for global retrieval; the verified paper should not claim full retriever replacement.
Does learned imported retrieval survive bridge checks?	Final-base8 averages 92.9% across four splits and reaches 53.5% imported bridge verified success versus 44.5% for learned+base fallback.	Bridge evidence supports the premise-recovery story but remains separate from open-ended proof generation.

A.3 FEATURE-GROUP ABLATION

Table 11: Feature-group ablation on 2k local trace-core goals. Static visible-feature reranking is weaker than blind expansion; failure-conditioned feature use is the important step.

Method	Success	FFR	Avg prem.
topk_expansion	91.5%	79.9%	46.9
visible_feature_name_rerank	85.3%	0.0%	42.9
visible_feature_statement_rerank	77.6%	0.0%	42.9
visible_feature_decl_rerank	78.3%	0.0%	42.9
visible_feature_rerank	86.5%	0.0%	42.9
rule_far_no_core_decl_features	92.6%	82.4%	46.2
rule_far_no_core_statement_features	92.8%	82.9%	46.4
rule_far_no_core_name_features	95.0%	88.1%	45.4
rule_far_no_core_tags	95.7%	89.7%	45.1

A.4 IMPORTED-CORE STRESS BEFORE LEARNED RETRIEVAL

Table 12: Initial imported-core stress test on 2k goals. This table is a diagnostic negative result: local and lexical signals are insufficient for global imported-premise recovery.

Method	Success	FFR	Avg prem.
one_shot	1.1%	0.0%	32.0
same_file_prior_rerank	2.1%	0.0%	56.0
visible_feature_rerank	2.4%	0.0%	56.0
bm25_rerank	7.0%	0.0%	56.0
leansearch_iterative	11.9%	8.3%	91.9
bm25_expansion	11.9%	8.3%	92.2
rule_far_bm25	12.2%	8.7%	92.2
rule_far_full (oracle ref.)	55.7%	55.2%	86.6

A.5 BRIDGE FAILURE TAXONOMY

The 200-goal imported bridge verifies 124/200 original traced tactic scripts. The primary replay failures are 30 unknown-identifier errors, 13 other Lean errors, 8 typeclass failures, 7 unsolved-goal failures, 6 simp no-progress failures, 5 tactic failures, 5 type mismatches, and 2 sorry-context failures. At the policy level, final-base8 has 17 replay-verified trace misses, compared with 19 for the original second-stage controller, 35 for learned+base fallback, and 47 for learned expansion. Negative-case inspection finds 0/17 candidate-pool misses, 2 fallback-selected core drops, 3 expansion-selected core drops, and 15 failure-conditioned rank misses. This supports the limitation that remaining headroom is ranking and calibration, not candidate generation.

A.6 GUARDRAIL AND EXPERT-MIXING ABLATIONS

The final base8 guardrail was selected because broader heuristics were negative. Hard base guardrails reduce bridge trace-core success from 72.0% to 63.0% or 60.0%. Soft base-score mixing is inert at low mixing weights and harmful at high weights. Imported-failure multi-expert routing drops bridge trace-core success to 62.0%. A larger final expert guardrail also underperforms final-base8 on the original split, fold0, and bridge subset. These results justify stopping hand-written expert-mix tuning and treating calibrated expert gating as future work.

B CLAIM-EVIDENCE SELF-REVIEW

Table 13: End-of-draft self-review. Claims in the abstract and introduction are aligned to reported evidence; unsupported stronger claims are explicitly avoided.

Dimension	Current answer	Residual risk
Contribution	Action-conditional evidence allocation is formulated as distinct from premise ranking, with frozen holdout and Aesop controls separating the core claim from mechanism diagnostics.	A fixed portfolio can look simple unless the typed-interface intervention remains central.
Writing clarity	Main text now leads with three verified layers: mixed-source matrix, retrieved-only anchor, and frozen fresh holdout; trace-core is kept as discovery evidence.	Method names are long; final draft may need shorter aliases.
Experiments	Results cover verified replayable Mathlib actions, budgeted allocators, retrieved-only compilation, frozen fresh-holdout folds, Aesop counterfactuals, trace-core discovery, and replay bridges.	Full action grid is still 230/490; holdout is retrieved-only and not a population-scale benchmark.
Evaluation	Kernel-verified action success, oracle headroom, fixed-budget policy, holdout transfer, and interface ablations are reported.	Full open-ended theorem proving and official retriever comparison are out of scope.
Soundness	Negative gates show broad insertion, strict syntactic filtering, and learned adaptive allocation are not enough.	Learned allocation does not yet beat the fixed typed control.